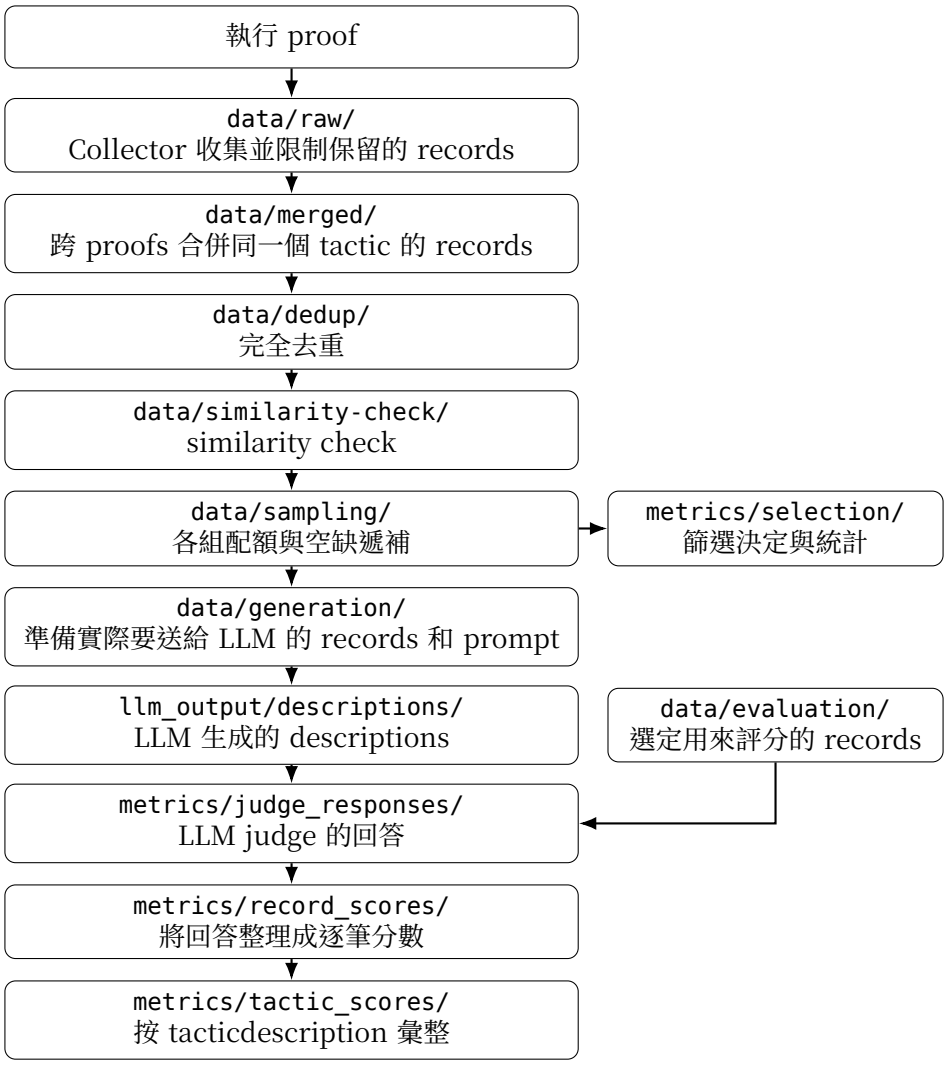
1 Framework

資料流程框架



跨階段追溯：`data/metadata/` 記錄上述各階段的來源、設定與檔案關係。

各階段資料夾說明

順序	資料夾	裡面存什麼
1	`data/raw/`	Collector 跑完 proof 後輸出的 records。已經受到 collector 的大小、容量與選取規則限制，因此不是執行過程中的所有 records。
2	`data/merged/`	把不同 proofs 中同一個 tactic 的候選 records 集合在一起，此階段仍可包含重複資料。
3	`data/dedup/`	依既有 record identity 規則，移除 完全重複後的 records。尚未做後面的 similarity 與最終配額選 擇。
4	`data/similarity-check`	經過 similarity check
5	`data/sampling/`	分組配額與遞補後，最後選出的 records, 供 description generation 使用。
5 的紀錄	`metrics/selection/`	為什麼保留或排除某筆 record、相似度比較、各組數量與篩選設定。這不是 LLM 對 description 的評分。

順序	資料夾	裡面存什麼
6	`data/generation/`	實際生成請求：包含送進 LLM 的 records、prompt 和相關設定。若生成時另外抽樣或截斷，這裡記錄實際使用的版本。
7	`llm_output/descriptions/`	LLM 產生的 descriptions，以及生成嘗試與狀態等紀錄。
8	`data/evaluation/`	Judge 使用的 records、候選 descriptions 的對應資訊與評分任務。放在這裡不代表它們與生成資料沒有重疊，是否獨立要看實驗設計。
9	`metrics/judge_responses/`	保存 judge 的回答，包括逐筆判斷、理由、用量和呼叫錯誤。
10	`metrics/record_scores/`	把有效回答整理成每筆 record 對每份 description 的分數與理由。目前這個 judge 的對應是 full=1、partial=0.5、none=0。
11	`metrics/tactic_scores/`	將逐筆分數彙整成每個 tactic/description 的結果，供比較、報表與表格使用。
全流程	`data/metadata/`	s2n-bignum commit、實驗設定、來源檔案、雜湊、搬移對照等追溯資訊。各資料清單的 index 也可以與 records 放在同一階段資料夾。

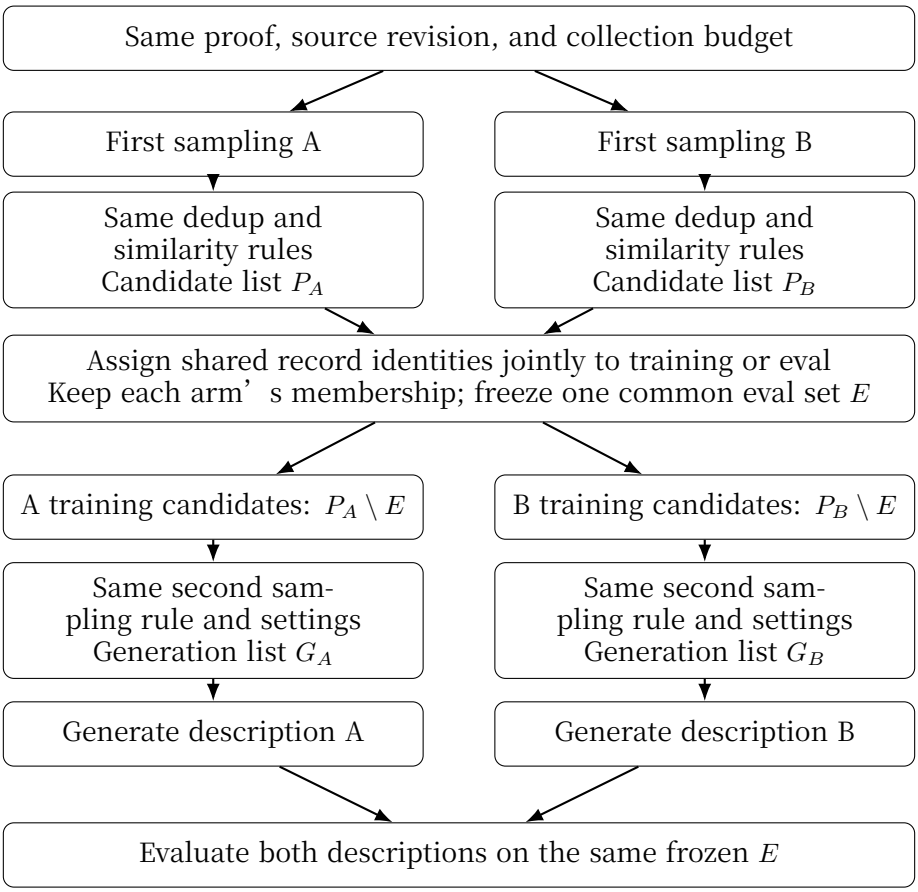
2 比較兩階段 sampling 時的 training 與 evaluation 切分

本節記錄預定的實驗設計，不代表已經實作或執行。 這裡的 training records 是提供給 LLM 生成 description 的範例，不會訓練或更新模型權重。 為避免混淆，下文區分「training 候選池」與「最後送進 LLM 的 generation list」。

- **第一次 sampling：**collector 在 proof 執行時選取並保留候選 records。
- **中間處理：**依既定流程合併候選、exact dedup 與 similarity check。
- **第二次 sampling：**從候選池依分組配額與遞補規則，選出最後用來生成 description 的 records。

2.1 情況一：比較第一次 sampling 方法

A、B 從相同 proof 與 source commit 開始，使用不同的第一次 sampling 方法。 兩邊使用相同候選容量，並固定其餘 collector 設定、dedup 與 similarity check 規則。 因此，中間處理後的候選 lists 可以不同，這正是要保留的實驗差異。



一起決定歸屬，不是混合兩邊的生成資料。 一起查看 A、B 的候選，是為了辨認內容相同的 records，統一決定它們是否
evaluation。 若某筆 record 被選入共同 eval set，必須從兩邊的 training 候選中排除。 不能將 B 獨有的 record
加進 A，或反過來補入。 A training 與 B training 可以互相重疊，但任一方都不能與 eval 重疊。
例如，下列編號代表依內容辨認的 record identity，而不是檔案內的行號：

$P_A = \{r_1, r_2, r_3, r_4\}$，$P_B = \{r_2, r_3, r_5, r_6\}$。
若選定 $E = \{r_3, r_5\}$，則
A training 為 $\{r_1, r_2, r_4\}$；
B training 為 $\{r_2, r_6\}$。

兩份 descriptions 都評 E，即使其中的 r_5 原本只存在於 B 候選池。 共同 eval 不限於 A、B 的交集，但每個內容
identity 只保留一份。

控制最後送進 LLM 的筆數。 在相同 tactic 內，對 success 與 failure 分開決定共同的生成筆數：

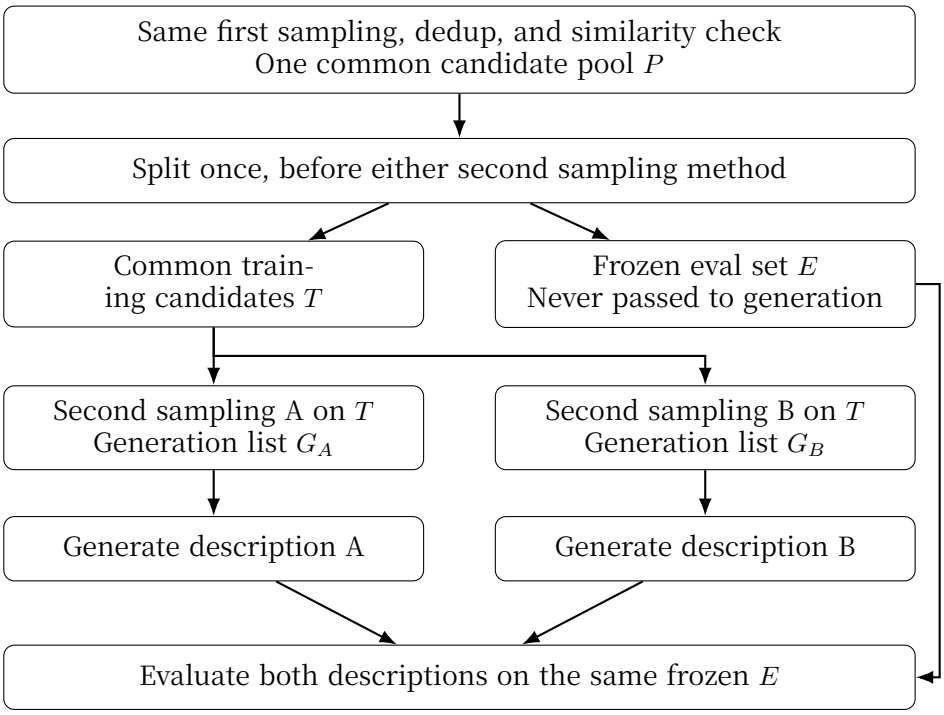
$$n_{t,o} = \min\big(K_{t,o},\ |T_{A,t,o}|,\ |T_{B,t,o}|\big),$$

其中 t 表示 tactic，o 表示 success 或 failure，T 是移除 eval 後的 training 候選，K 是預定的生成上限。 兩邊各自用相同的
deterministic 第二次 sampling 規則選出 $n_{t,o}$ 筆。 這個設計對齊的是每個 tactic 的 success／failure 總筆數，
不額外要求各 goal-count group 的實際筆數都相同。 相同筆數也不代表相同 token 數或 LLM 費用。 若某個 out-
come 在任一邊沒有 training 候選，對齊後兩邊該 outcome 都使用零筆；應明確記錄此限制。
例如，若 success 上限為 40，兩邊可用候選分別為 35 與 28，最後各選 28 筆。 較多的一方不必刪除其餘已保存候選；
它們只是未用於這輪生成。 這是**固定生成筆數的比較**，不是讓每種方法用盡所有可用資料的比較。

第一次先保留較大的候選池。 若擔心切出 eval 後 training 太少，可以在第一次 sampling 使用相同且較大的候選容量。
例如，兩邊先各保留 120 筆候選，經過去重、similarity 與 eval 排除後，再各選最多 40 筆 success。 這只是示例，
不是已決定的參數，也不保證每個 tactic 都有足夠資料。 先離線統計各階段保留量，再決定容量與 eval 大小；不能複製
records 湊數或將 eval 放回 training。 候選容量增加也會增加收集與儲存成本。

2.2 情況二：比較第二次 sampling 方法

第一次 sampling、dedup 與 similarity check 全部相同，先取得一份共同候選池 P。 **在第二次 sampling 之前只切分一次，**
得到共同 training 候選池 T 與固定 eval set E。 A、B 的第二次 sampling 都從同一份 T 開始選。



例如，共同候選池有 100 筆 success，先保留 20 筆作為 eval，剩下 80 筆作為 training 候選。 A、B 各自從同一份 80 筆候選選 40 筆，生成兩份 descriptions，然後都評同一份預留的 20 筆。 這些數字只是示意，failure 也需要另外規劃保留不建議先執行第二次 sampling，再從 A、B 的最終 lists 抽走 eval，然後又 sampling 一次。 這會移除方法刻意保留的代表，測到的變成兩輪選取與移除的整體效果。 若要單獨比較第二次選取方法，應固定共同 training 候選、生成上限與非比較中的若某個方法因組別限制無法填滿共同目標，須事先決定如何處理並記錄實際筆數，不能默認兩邊一定選滿。

2.3 兩種設計都必須遵守的規則

令 G_A、G_B 為實際送進 LLM 的 generation lists，共同的無重疊要求為

$$G_A \cap E = \emptyset, \qquad G_B \cap E = \emptyset.$$

不要求 $G_A \cap G_B = \emptyset$：兩種方法可以選中相同的 training records。

1. **按內容決定歸屬**。 不能只比較檔名、proof 名稱或 record 編號。 應依一致的內容 identity 規則辨認 records，並檢查生成與評分實際看到的 evidence 是否重複；僅來源 metadata 不同不能當成新的獨立案例。
2. **先切分、固定，再生成與評分**。 切分採 deterministic 規則，保存 membership 與設定，不依生成結果或 judge 分數重新挑選 eval。 切分比例、穩定排序規則、各 tactic／outcome 最低筆數仍待決定。
3. **不足資料時明確記錄**。 分別記錄各 tactic 的 success／failure 與 goal-count 分布。 若沒有可評 records，該項標記 N/A；若生成資料不足，依事先約定的規則縮小規模或不納入，不把缺失資料當成零分。
4. **固定生成與 judge 條件**。 除了要比較的 sampling 方法，固定 prompt、模型、effort、token budget 與評分規則。 兩份 descriptions 評同一份 eval evidence，分別呈現每個 tactic 的 success／failure 分數與筆數。 Deterministic 選取不代表 LLM 回答必然相同。
5. **保存每階段數量與來源**。 保存 source commit、collector／selector 版本、候選設定、去重與 similarity 設定、split membership、最終 generation lists 與固定 eval set。 報告原本可用量及最後實際使用量，避免將筆數差異誤解為內容選取的改善。

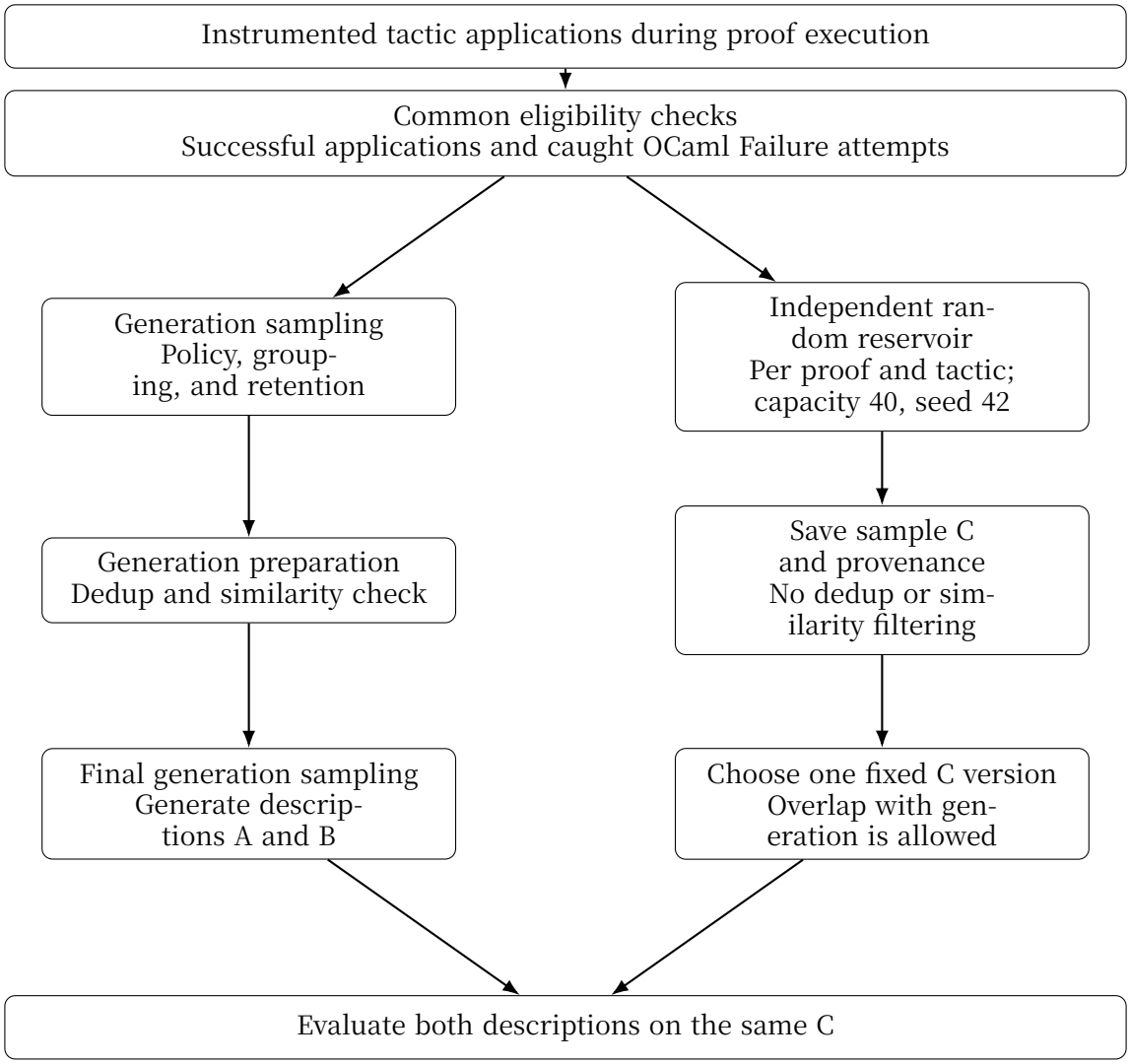
解讀限制。 這是同一批 proof records 內的 held-out evaluation，不是對另一個未見 proof 的泛化測試。 完全不同的 records 也可能非常相似；若需要更嚴格的分離，可將相似 records 以固定共同規則分成群，整群分到同一邊，但此項尚未決此外，切分發生在 similarity check 之後，共同 eval 的分布仍受上游收集與篩選影響，不能將結果直接推廣到所有 proof 執行案例。

3 在 proof 執行時收集獨立隨機 evaluation set C

本節說明另一種已實作的資料收集流程：在 proof 執行時，從符合條件的 tactic applications 另外抽出 evaluation set C。 這裡的「獨立」指抽樣途徑不依賴 generation sampler 的保留結果，不表示 C 與生成資料必然沒有重疊。 本節依目前程式與文件說明流程，不代表已完成完整 proof 實驗或 description 品質評估。

3.1 先分流，再分別建立生成資料與 C

C 的分流發生在 generation sampling、dedup 與 similarity check 之前。 每次被 instrumentation 記錄的 tactic application，通過共同的大小與 assumption 數量限制後，同時可進入 generation 路徑與獨立的 random reservoir。因此，即使某筆 record 後來沒有被 generation sampler 保留，它仍有機會進入 C。若只從已保存的 generation records 離線再抽樣，就無法找回執行時已丟棄的 records，因而不是這裡的流程。



圖中左側代表 A、B 各自的生成流程；要比較第一次 sampling 時可使用不同 collector policy，比較第二次 sampling 時則固定共同候選池。右側的 C 不從 A、B 的最終 generation lists 選取，兩份 descriptions 必須使用同一版本的 C 評分。

哪些 applications 會進入 C？ 成功的 tactic applications 與被捕捉為 OCaml `Failure` 的失敗嘗試都可納入，包括成功 proof 中途試過但失敗的 tactic。Conversions 與其他種類的 exceptions 不當成這裡的 tactic failure。共同 eligibility 限制仍然存在，因此 C 代表符合條件且有 instrumentation 的 applications，不能視為所有 proof states 或 HOL Light 所有操作的完整樣本。

抽樣單位與預設設定。 每個 proof execution 的每個 tactic 各有一個 random reservoir，預設最多保留 40 筆，independent seed 為 42。Success 與 failure 共用這 40 筆容量，不按 outcome 或 goal count 設配額，所以不保證兩種 outcome 都有樣本。每次出現算一個 occurrence：內容相同的兩次呼叫仍是兩次抽樣機會，也可能同時被保
相同 seed 與相同順序的 eligible stream 可重現選取結果；執行順序改變則結果可能改變。不同 proofs 使用相同 seed，也不保證它們的抽樣在統計上彼此獨立。

這不是 held-out evaluation。 本節允許 $G_A \cap C$ 或 $G_B \cap C$ 非空，其中 G_A、G_B 是實際用於生成 descriptions 的 records。不要因為重疊而補抽或移除 C，否則會改變原先的抽樣分布。有保存實際 generation requests 時，可以量測並回報 overlap。前一節要求 generation 與 eval 無重疊，本節則評估固定 eligible stream 樣本上的表現；兩種設計的評分意義不同，不能將本節結果稱為未見 records 的泛化表現。

3.2 日常執行：自動沿用，或收集新的 C

日常 sampling workflow 與新建立的 experiment config 預設使用 `auto`。檢查發生在啟動實驗時，依經過驗證的 build 所使用的 source 決定能否沿用 C；既有 pinned-source 設定仍決定實際執行哪個版本。這不是背景檔案監看器，修改或新增 `.ml` 檔案本身不會立刻觸發收集。

- **auto：**找到來源與獨立抽樣設定相符、且通過驗證的 sample 時沿用；新 proof 或相關 inputs 改變、沒有相符 sample 時，在 proof 執行中收集新的 C。沿用 C 時 generation 的 proof execution 仍會執行，只是該 proof 的獨立 collector 關閉。
- **on：**強制在這次新的 proof execution 收集，適合需要目前 source revision 對應資料的情況。
- **off：**這次執行不收集也不沿用 C。Independent capacity 設為零時，在所有模式下都停用此功能。

是否能沿用，不只檢查最外層 proof `.ml`：還包含展開後的 source dependencies、HOL source、proof 以 literal 路徑引用的 object files，以及獨立抽樣的規則、capacity 與 seed。程式不推測執行時才計算出的 object 路徑。只改 generation policy、分組、配額、filtering、最終選取或 description prompt，本身不要求重抽 C；unrelated proof 的新增 object 也不會使此 sample 失效。Independent seed 與 generation 的 `--seed` 分開設定。
手動覆寫可使用 `--independent-eval-mode`、`--independent-eval-capacity` 與 `--independent-eval-seed`。較底層的 `scripts/run-with-sampling.sh` 仍是手動流程，不會自動搜尋或發布可沿用的 C；舊的 explicit capacity／seed configs 也保留原有手動行為。歷史 schema-v1 configs 除非明確啟用，否則維持停用。新的 collector 與 runtime adapter 必須編入 executable；只更新腳本不會更新既有 `.native`。

3.3 保存版本、新增 proof，以及評分時的來源限制

Collector 將 records 與 metadata 分別寫到`PROOF.ml.independent-eval/TAC.json` 與`PROOF.ml.independent-eval.json`。Metadata 保存 seed、capacity、eligible／retained 數量與選中 occurrence 的位置，workflow 再附上 build 與 source provenance。這些 records 不會進入一般 generation preparation。
自動流程在本機為每個 proof 保存一份可驗證的 sample，並在 collection receipt 的 `independent_eval.reference` 記錄引用。發布檔案位於 dataset root 的 `data/evaluation/` 與`data/metadata/`；這個動作不會上傳 Hugging Face。改變來源後產生新 sample，舊版本與原始 provenance 都保留。沿用舊 sample 也不會把它的 source commit 改寫成新 commit。

新增 proof 不會自動合併成一份全域 C。 將新 proof 加入選取清單後，`auto` 可沿用未變更 proofs 的 samples，並收集新 proof 中既有與新出現 tactics 的樣本。各 proof references 仍然分開；需要共同評分集合時，使用手動 publisher 明確合併指定 samples。可用 `--previous-manifest` 保留舊 samples 並加入新 proof，建立新的 C 版本，不覆寫舊版本也不再隨機抽一次。同一 proof、同一 source version 的重複匯入會被拒絕。A、B 必須一起改用相同的新版本。
這是逐 proof、逐 tactic 抽樣後的累積，不是把所有 proofs 的 occurrences 合在一起均勻抽樣。例如，同一 tactic 在兩個 proofs 各有 20 與 200 個 eligible occurrences，capacity 40 時前者全留，後者只留 40 筆，兩邊單筆 occurrence 的入選機率不同。報告分數時應一併列出 proof 清單、版本、各 tactic 的筆數與 outcome coverage。

沿用判斷與 judge 的 commit 檢查不同。 來源內容相同時，自動流程可能跨 Git commits 沿用 sample，但目前 judge 要求 C 與兩份 description setups 共用同一個已知 source commit。因此跨 commit 沿用的 sample 仍可能被 judge 拒絕。需要對齊時，使用 `on` 在所需 revision 收集，再手動發布這次輸出；合併評分用 proofs 也應來自同一 revision。保留 raw samples、build receipts 與 manifests，不要為通過檢查而改寫舊資料的來源。